O(1) Lite — Explanation Guide

How to explain the database design, concept by concept, for the DBMS project evaluation and viva. Read this together with `DB-Design-Lite.pdf` (the formal submission); section numbers like §3.4.6 point into that document.

1. The project in 60 seconds

> *"Our college ICPC teams used a spreadsheet to track who solved which Codeforces problem. Every row repeated the student's details and the problem's details, so it had insertion, update and deletion anomalies. We took that spreadsheet as our starting relation, found its functional dependencies, computed the keys with attribute closure, reduced the FDs to a minimal cover, and normalised it to 3NF — actually BCNF. We proved the decomposition is lossless with the chase test and dependency preserving. The result is nine tables in PostgreSQL with constraints, a trigger and views."*

2. The nine tables in one line each

Table	Key	What it stores / why it exists
University	univ_id	University name and city — separated to remove the transitive dependency user → univ_id → univ_name.
Team	team_id	ICPC team name — separated to remove user → team_id → team_name.
User	user_id (also email)	Students. team_id is a nullable FK (a student is in at most one team).
Topic	topic_id	Topic names (DP, Graphs…), typed once.
Problem	problem_code	Codeforces problem, title, rating. Separated because of the partial dependency problem_code → title, rating.
Problem_Topic	(problem_code, topic_id)	Which topics a problem has — this is how we fixed the 1NF violation (topics was a list).
Submission	(user_id, problem_code, submitted_at)	Every attempt with its verdict. A **weak entity** in the ER diagram.
Sheet	sheet_id	A team's practice list, with a deadline.
Sheet_Problem	(sheet_id, problem_code)	Problems in a sheet, in order. Has a second candidate key (sheet_id, position).

Plus three **views** (difficulty band, leaderboard, sheet progress) and one **trigger** (max 3 members per team).

2b. "Why a database? Can't you just call the Codeforces API?"

This is the most likely first question. The short answer:

> *"The API gives one student's raw submissions. Teams, sheets, topics and universities are our own data, which no API has. And the leaderboard and team progress need many students' data joined together, which is a SQL query, not 300 API calls."*

Need	With only API calls	With our database
Teams, sheets, topics, universities	Impossible — Codeforces doesn't know about them	Stored in our tables
Leaderboard of 300 students	300 API calls on every page load, and the API is rate-limited	One GROUP BY query (v_leaderboard)
Team progress on a sheet	Call the API for each member, then match problems by hand in code	One JOIN (v_sheet_progress)
"Team of max 3", "no duplicate problem in a sheet"	No guarantee; any bug can break it	Trigger and UNIQUE constraint enforce it
Codeforces is down	App is down	App still works with stored data

The API is only the **input** for one table (Submission, filled by a sync job). Everything else, including every query the app runs, is DBMS.

3. Suggested presentation flow (≈10 minutes)

1. **Problem statement** (1 min) — the spreadsheet and its pain.
2. **ER diagram** (2 min) — entities, the weak entity Submission, cardinalities, total participation, derived attribute.
3. **Anomalies** (1 min) — show the 5-row spreadsheet and point at one example of each anomaly.
4. **FDs → closure → keys** (2 min) — list F, compute $\{U,P,S\}^+$ live on the board.
5. **Minimal cover** (1 min) — the 3 steps; mention it is not unique.
6. **1NF → 2NF → 3NF** (2 min) — one violation and one fix at each step.
7. **Lossless + dependency preserving** (1 min) — chase table, and the spurious-tuple counter-example.
8. **Live demo** — run a query or two on PostgreSQL and try to add a 4th team member (the trigger rejects it).

If you are presenting as a group, split it as 1–2 / 3–5 / 6–7 / 8 so each person owns a part.

4. Concepts, one by one

For each concept: what it means in plain words, where it shows up in our project, and a line to say in the viva.

4.1 Keys

- **Super key** — any set of attributes that uniquely identifies a row. (user_id, name) is one.
- **Candidate key** — a *minimal* super key: remove any attribute and it stops being unique. User has two: user_id and email.
- **Primary key** — the candidate key we pick. **Alternate key** — the ones we did not pick (email, enforced with UNIQUE).
- **Composite key** — a key of more than one attribute: Submission's (user_id, problem_code, submitted_at).
- **Foreign key** — a column that must match a key in another table: User.univ_id → University.

> *"cf_handle is UNIQUE, but it is not a candidate key, because it can be NULL — a key must identify every row."*

4.2 ER model

- **Weak entity** — cannot be identified by its own attributes. A Submission is only identified by *which user*, *which problem* and *when* (partial key: submitted_at). Drawn as a double rectangle, with double-diamond identifying relationships.
- **Cardinality** — 1:N (a university has many users), M:N (problems ↔ topics, sheets ↔ problems).
- **Participation** — total (double line): every sheet must belong to a team; partial: a student need not be in a team.
- **Derived attribute** — difficulty is computed from rating, so it is dashed in the diagram and is a view, not a column.
- **Mapping to tables** — 1:N becomes a foreign key on the N side; M:N becomes a junction table (Problem_Topic, Sheet_Problem).

4.3 Anomalies — why normalise at all

Anomaly	Example in the spreadsheet
Insertion	Can't add a new problem until somebody submits it.
Update	Saharsh's e-mail is on 3 rows; change one, forget the others → inconsistent.
Deletion	Delete Anu's only attempt at 1547G → we lose that 1547G exists at all.

4.4 Functional dependency (FD)

X → Y means "if two rows have the same X, they must have the same Y". user_id → name: the same user id can never have two names. **Trivial**: the right side is already in the left (UM → U). Our letters: U user_id, N name, M email, I univ_id, J univ_name, G team_id, K team_name, P

problem_code, T title, R rating, S submitted_at, V verdict, L language.

Armstrong's axioms (memorise the three, know the three derived ones):

Rule	Statement	Our example
Reflexivity	$Y \subseteq X \Rightarrow X \to Y$	`UP → P`
Augmentation	$X \to Y \Rightarrow XZ \to YZ$	`P → T ⇒ PS → TS`
Transitivity	$X \to Y, Y \to Z \Rightarrow X \to Z$	`U → I, I → J ⇒ U → J`
Union	$X \to Y, X \to Z \Rightarrow X \to YZ$	`P → T, P → R ⇒ P → TR`
Decomposition	$X \to YZ \Rightarrow X \to Y$	`U → NMIG ⇒ U → N`
Pseudo-transitivity	$X \to Y, WY \to Z \Rightarrow WX \to Z$	`M → U, UPS → VL ⇒ MPS → VL`

4.5 Attribute closure X^+

"Everything you can reach from X." Start with X, keep firing any FD whose left side you already have, stop when nothing new appears.

`{U,P,S}+: UPS → +NMIG → +J → +K → +TR → +VL = all 13 ⇒ super key.`

Uses: (1) find keys, (2) test if an FD holds ($X \to Y$ holds iff $Y \subseteq X^+$), (3) minimal cover steps.

> *"Shortcut to find keys: P and S never appear on the right side of any FD, so they must be in every key. $\{P,S\}^+$ is only {P,S,T,R}, so we add U or M — giving the two candidate keys {U,P,S} and {M,P,S}."*

4.6 Closure of FDs F^+

F^+ is every FD that follows from F. It is huge, so we never write it out. We show members by deriving them with Armstrong's axioms ($U \to J$, $M \to K$) and we test membership with attribute closure ($\{I\}^+ = \{I, J\}$, so $I \to U$ is *not* in F^+). Two FD sets are **equivalent** if they have the same closure — that is what makes the minimal cover "the same rules, fewer of them".

4.7 Minimal cover

1. **Split** right sides: U → NMIG becomes U → N, U → M, U → I, U → G.
2. **Extraneous left attributes**: in UM → N, $\{U\}^+$ already has N, so M is extra → U → N. In UPS → T, $\{P\}^+$ has T → P → T.
3. **Redundant FDs**: U → J can be derived (U → I → J) without itself → delete.

F_C `= { U → NMIG, M → U, I → J, G → K, P → TR, UPS → VL }`

> **Trap question:** "Is the minimal cover unique?" — No. In UM → N both U and M are individually extraneous; removing U first gives M → N instead of U → N. Both are valid minimal covers.

4.8 Normal forms

NF	Rule (simple words)	Violation in our data	Fix
1NF	Every cell has one atomic value.	topics = "DP, Graphs"	Problem_Topic table, one row per pair
2NF	1NF + no non-prime attribute depends on *part* of a key.	U → name (U is part of key UPS); P → title	Split out USER_INFO (U^+) and PROBLEM (P^+)
3NF	2NF + no non-prime attribute depends on another non-key attribute (no transitive dependency). Formally: for X → A, X is a super key or A is prime.	U → I → J, U → G → K	Split out UNIVERSITY(I,J) and TEAM(G,K)
BCNF	For every X → A, X is a super key (no exceptions).	none	already satisfied

> *"M → U does not break 2NF even though M is part of the key {M,P,S}, because U is a prime attribute. 2NF and 3NF only care about non-prime attributes."*

> *"The 3NF synthesis algorithm on our minimal cover gives exactly the same five tables, and synthesis is guaranteed lossless and dependency preserving — so our two proofs are a cross-check."*

4.9 Lossless join

Splitting a table is safe only if joining the pieces gives back exactly the original — no missing rows and no fake (**spurious**) rows.

- **Two-way test:** the common columns must be a key of at least one piece. USER_INFO and the rest share U, and U is the key of USER_INFO ✓.
- **Chase / tableau test** (for many pieces): a table with one row per relation, **a** where the relation has the column. Apply FDs to equalise values. A row that becomes all **a** ⇒ lossless. Our SUBMISSION row becomes all **a** (§3.4.10).
- **Counter-example to show:** split Submission into (user, problem) and (problem, time, verdict, lang). Joining back invents "Saharsh submitted 455A at 19:05 in Python" — he didn't.

4.10 Dependency preservation

After splitting, can each FD still be checked inside one table? Ours: U → NMIG and M → U live in User, I → J in University, G → K in Team, P → TR in Problem, UPS → VL in Submission. All preserved ✓.

Counter-example: store univ_name against each user in a separate table Y(U, J). Now I → J is in no table; the algorithm gives result = {I} and never reaches J ⇒ not preserved. The DB could hold two names for the same university id.

> **Trap question:** "Is every BCNF decomposition dependency preserving?" — No. 3NF can always be both lossless and dependency preserving; BCNF is always achievable losslessly but sometimes loses a dependency. In our case we got lucky: BCNF and preserving.

4.11 Constraints, trigger, views, SQL

- **Domain constraints** — CHECK (rating BETWEEN 800 AND 3500), verdict must be one of OK, WRONG_ANSWER, ….
- **Entity integrity** — primary keys are NOT NULL and unique. **Referential integrity** — foreign keys; ON DELETE CASCADE removes a deleted user's submissions.
- **Trigger** — `team_size_check` raises an error on a fourth team member. A CHECK constraint cannot do this because it would have to count other rows.
- **Views** — v_leaderboard, v_sheet_progress, v_problem_difficulty. Derived data is computed, never stored, so it can never go out of date.
- **Index** — `university_name_trgm` is a GIN trigram index (pg_trgm). Sign-up uses `similarity()` on it to suggest "Amrita Vishwa Vidyapeetham" when someone types "amritha", so the same college is not inserted twice.
- **Team analytics** — uses a CTE (`WITH first_solves AS …`), window functions, and the "gaps and islands" trick for streaks: for consecutive active days, `day − row_number()` is constant, so grouping by it gives each unbroken streak.
- **Queries** in the appendix show JOIN, LEFT JOIN, GROUP BY / HAVING, NOT EXISTS subquery, M:N through a junction table, and a window function (rank).

5. Viva questions with answers

Question	Answer
Why not just keep one big table?	Redundancy causes insertion, update and deletion anomalies (give the spreadsheet examples).
What are the candidate keys of the practice log?	{user_id, problem_code, submitted_at} and {email, problem_code, submitted_at}.
How did you find them?	Attribute closure. P and S are never on a right side so they're in every key; adding U or M makes the closure all attributes, and no smaller set works.
What are prime attributes here?	U, M, P, S — those in some candidate key.
Difference between 2NF and 3NF?	2NF removes partial dependencies (on part of a key); 3NF removes transitive dependencies (non-key → non-key).
Difference between 3NF and BCNF?	3NF allows X → A when A is prime even if X is not a super key; BCNF does not. All our tables are BCNF.
Show a transitive dependency.	user_id → univ_id → univ_name.
Show a partial dependency.	problem_code → title, where the key is (user_id, problem_code, submitted_at).
What is an extraneous attribute?	A left-side attribute you can drop without changing the closure: M in UM → N.
How do you know a decomposition is lossless?	Common attributes are a key of one side, or the chase produces an all-a row.
What is a spurious tuple?	A fake row produced by joining a lossy decomposition.
Why is Submission a weak entity?	It has no identifier of its own; it's identified by its user and problem plus submitted_at (partial key).
Why is difficulty not a column?	It is derived from rating; storing it adds the transitive FD rating → difficulty and can go stale. It's a view.
Why a trigger for team size?	The rule involves counting other rows, which a CHECK constraint cannot do.
Why is team_id in User and not a separate membership table?	A student is in at most one team, so it is an N:1 relationship → foreign key on the N side; nullable because participation is partial.
Why use a DBMS if Codeforces has an API?	The API only gives raw submissions per handle. Teams, sheets, topics and universities are our own data; leaderboard and progress need joins across many users; constraints and the trigger keep the data valid. The API just feeds the Submission table.
How is the streak	Gaps and islands: number each active day with row_number(); for consecutive

calculated?	days, day − row_number is the same, so GROUP BY that value gives each streak and count(*) its length.
Why store cf_rating if derived values are not stored?	It is not derivable from our tables; it comes from Codeforces' contest history. It is a snapshot that depends only on user_id, so the table stays in BCNF.
What is the trigram index for?	Fuzzy "did you mean" college search. similarity() compares names by shared 3-letter chunks, and the GIN index makes it fast. It stops duplicate colleges like "Amrita" and "Amritha".
Why did you reduce from the full design?	The 20-table version had many tables that repeated the same concepts; nine tables cover every concept and are easier to verify.

6. One-page cheat sheet

Item	**Value**
Universal relation	`PRACTICE_LOG(U N M I J G K P T R S V L)`
Given F	`U→NMIG, M→U, I→J, G→K, P→TR, UPS→VL, UM→N, U→J, UPS→T`
Minimal cover	`U→NMIG, M→U, I→J, G→K, P→TR, UPS→VL`
Candidate keys	`{U,P,S}, {M,P,S}`
1NF fix	topics list → Problem_Topic
2NF split	`USER_INFO(U,N,M,I,J,G,K), PROBLEM(P,T,R), SUBMISSION(U,P,S,V,L)`
3NF split	`USER(U,N,M,I,G), UNIVERSITY(I,J), TEAM(G,K)`
Lossless?	Yes — chase gives all-a SUBMISSION row
Dependency preserving?	Yes — every FD of F_c sits in one table
Final normal form	BCNF (all 9 tables)

7. Files

File (docs/lite/)	**Purpose**
DB-Design-Lite.pdf / .docx	The formal design document to submit.
Explanation-Guide.pdf / .docx	This guide.
schema-lite.sql	Tables, trigger and views (PostgreSQL).
seed-lite.sql	Sample data used in all examples.
queries-lite.sql	Demo queries for the evaluation.

er-lite.dot / .png / .svg	ER diagram source and renders.
build.sh	Rebuilds the PDFs and DOCX files.

Demo commands: `psql -d o1 -f schema-lite.sql -f seed-lite.sql -f queries-lite.sql`. The original full design in `docs/` is unchanged.